Hito 3 — Interfaz dinámica y persistencia

- Carrera: Tecnicatura Superior en Desarrollo de Software
- Institución: Instituto Superior de Formación Técnica Leopoldo Marechal
- Asignatura: Prácticas Profesionalizantes I
- Profesor: Alexander Seling
- Ayudante de Cátedra: Jorge Marinoni
- Septiembre 2026

1. Descripción

En el Hito 2 desarrollaron un prototipo navegable de su sistema, utilizando HTML semántico y CSS. En esta etapa comenzaremos a incorporar **comportamiento y datos dinámicos mediante JavaScript**.

El objetivo es transformar parte del contenido estático de la interfaz en información que sea **cargada, procesada y mostrada dinámicamente** a partir de datos estructurados.

No se busca desarrollar un backend ni una base de datos real. El objetivo es simular el funcionamiento de una aplicación utilizando **manipulación del DOM, modelado de datos en JSON,** `fetch()` **y** `localStorage`.

2. Requisitos

1. **Modelar los datos del dominio** mediante objetos y arrays.
2. Crear un archivo `data/datos.json` con los datos iniciales de la aplicación.
3. Incluir en `datos.json` **al menos 2 usuarios por cada rol** definido para el sistema.
4. Incluir **al menos 2 tipos de datos relacionados**, con relaciones suficientes para demostrar su funcionamiento.
   Ejemplo: 2 médicos, 2 pacientes y 2 turnos para cada paciente.
5. Utilizar `fetch()` para cargar los datos desde `data/datos.json`.
6. Utilizar `localStorage` para conservar los datos modificados por el usuario y la información del usuario conectado, manteniendo la persistencia durante el uso de la aplicación.
7. Reemplazar parte de la información hardcodeada en el HTML por **contenido generado dinámicamente mediante JavaScript**.
8. Implementar un **inicio de sesión simulado** utilizando los usuarios definidos en `datos.json`.
   No se requiere desarrollar un backend, una base de datos SQL ni un sistema de autenticación real.
9. Adaptar la interfaz y las funcionalidades según el **rol del usuario conectado**.

10. Hacer que los formularios que representen operaciones sobre los datos **modifiquen el estado almacenado en** `localStorage`.
11. Reflejar las modificaciones realizadas en las distintas vistas de la aplicación.
    Ejemplo: un turno creado mediante un formulario deberá aparecer posteriormente entre los turnos del usuario.

3. Datos esperados en `datos.json`

Los datos propios del dominio no deberán estar hardcodeados en el HTML.

Deberán estar definidos en `datos.json` y ser utilizados por JavaScript para generar la información correspondiente en la interfaz.

Ejemplo: nombres de usuarios, productos, turnos, reservas, publicaciones, etc.

4. Organización del proyecto

El proyecto deberá mantener la estructura de archivos y carpetas utilizada en el Hito 2.

Se incorporará una carpeta `data` que deberá contener el archivo:

```
data/
└── datos.json
```

El archivo deberá contener los datos necesarios para que la aplicación pueda mostrar y gestionar la información correspondiente al usuario conectado y al dominio del proyecto.

5. Limitaciones técnicas

5.1 JavaScript y Web APIs

Para este hito deberán utilizar los siguientes recursos:

- manipulación del DOM, `document.createElement()`, `document.querySelector()`, etc.
- `fetch()`
- métodos de arrays, como `forEach()`, `map()`, `filter()`, etc.
- localStorage: * `localStorage.getItem()`, `localStorage.setItem()`
- JSON: `JSON.parse()` y `JSON.stringify()`

5.2 Restricciones

En este hito no deberán utilizarse:

- frameworks;
- librerías externas;
- plantillas prediseñadas;
- bases de datos;
- servicios externos de autenticación.

6. Fecha y modalidad de entrega

Fecha límite de entrega: 27/09/2026 — 23:59 hs.

La entrega se realizará mediante el Aula Virtual del Campus, donde deberán enviar **únicamente el enlace al repositorio de GitHub** correspondiente al proyecto.

Se recomienda continuar utilizando el mismo repositorio que el hito anterior.